

OO Design: Parking Lot

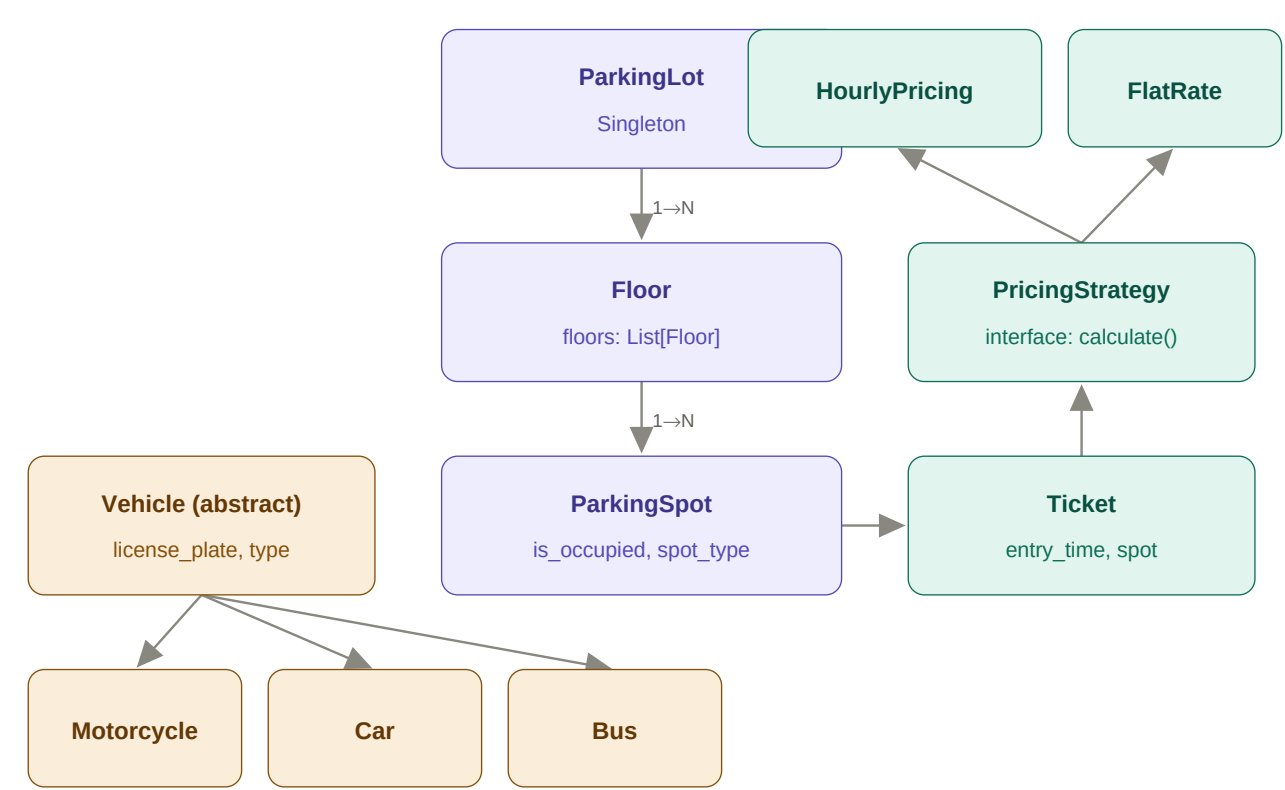
Object-Oriented Design — Senior Engineer Interview Walkthrough

1. Requirements & Class Identification

Think out loud: "Multi-floor, 3 vehicle types, hourly pricing, no reservation. Let me identify the nouns first: ParkingLot, Floor, Spot, Vehicle, Ticket, Receipt, PricingStrategy."

2. Class Diagram

Think out loud: "I'll draw the class relationships. ParkingLot owns Floors, Floors own Spots, Spots are assigned to Vehicles, Tickets link everything."



Class diagram: ParkingLot (Singleton) → Floor → ParkingSpot. Vehicle hierarchy (abstract base). Strategy Pattern for PricingStrategy.

3. Key Design Decisions

Compatibility rule: `can_fit(vehicle)`

This method lives on **ParkingSpot**, NOT on **ParkingLot**. Single Responsibility: the spot knows its own capacity. Rule: SMALL accepts [MOTORCYCLE]; MEDIUM accepts [MOTORCYCLE, CAR]; LARGE accepts all.

Why this choice: Putting `can_fit()` on `Spot` means we never need `isinstance()` checks in `ParkingLot.park_vehicle()`. The lot just asks each spot — polymorphism handles the rest. Open for extension: add `SpotType.HANDICAP` without touching existing code.

Strategy Pattern for pricing

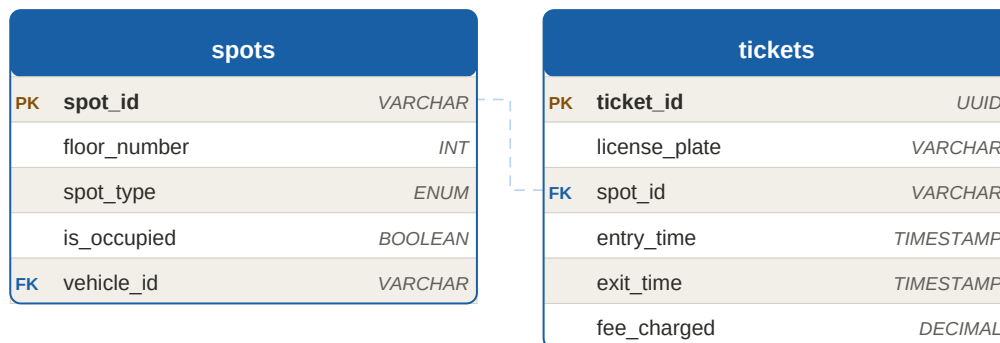
`PricingStrategy` is an interface with one method: `calculate(entry, exit, vehicle_type) → float`. Concrete implementations: `HourlyPricing`, `FlatRatePricing`, `WeekendSurgePricing`.

Why this choice: We can inject different strategies at runtime without modifying `Ticket`. On weekends, swap to `WeekendSurgePricing`. On holidays, use `HolidayPricing`. Zero code change in `Ticket` or `ParkingLot`. This is Open/Closed Principle.

Concurrency: thread safety on spot assignment

Race condition: two cars arrive simultaneously, both see `spot.is_occupied=False`, both call `assign()`. Solution: `ReentrantLock` per spot (single server) or `Redis SETNX` with `TTL` (distributed).

4. DB Schema (if persistence needed)



Schema: spots tracks occupancy. tickets logs every park/unpark event for billing and audit.

5. SOLID Principles Applied

- S — Single Responsibility: Spot manages occupancy; Ticket computes fee; PricingStrategy calculates price
- O — Open/Closed: Add `WeekendPricing` without modifying `Ticket`
- L — Liskov: Motorcycle/Car/Bus substitutable wherever `Vehicle` expected
- I — Interface Segregation: `PricingStrategy` exposes only `calculate()`
- D — Dependency Inversion: `Ticket` depends on `PricingStrategy` interface, not `HourlyPricing` concrete class